Dynamo vLLM Kubernetes Intel XPU Gap Analysis

Contents

Dynamo vLLM Backend (Kubernetes `DynamoGraphDeployment`): Intel XPU Support Gap Analysis

> Scope: **Kubernetes deployment only** (`examples/backends/vllm/deploy/`). The bash/CLI `launch/` scripts are out of scope for this document.
>
> Cross-checked directly against the `dynamo` repo at `examples/backends/vllm/deploy/` and `examples/backends/vllm/deploy/xpu/`.

✅ Already supported on Intel XPU today

These 8 have a real, shipped `deploy/xpu/*.yaml` Kubernetes DRA template in the repo — documented in full in `whitepaper/dynamo/01-aggregated.md` through `08-disaggregated-kv-router.md`:

Case	File
Aggregated	`deploy/xpu/agg_xpu_dra.yaml`
Aggregated + Tracing	`deploy/xpu/agg_tracing_xpu_dra.yaml`
Aggregated + KV Router	`deploy/xpu/agg_router_xpu_dra.yaml`
Aggregated + KV Router (no KV events)	`deploy/xpu/agg_router_kv_approx_xpu_dra.yaml`
Disaggregated	`deploy/xpu/disagg_xpu_dra.yaml`
Disaggregated + Tracing	`deploy/xpu/disagg_tracing_xpu_dra.yaml`
Disaggregated + Planner	`deploy/xpu/disagg_planner_xpu_dra.yaml`
Disaggregated + KV Router	`deploy/xpu/disagg_router_xpu_dra.yaml`

Plus, outside this specific directory: Global Planner (`examples/global_planner/`) and the heterogeneous Qwen3-VL Intel XPU + NVIDIA GPU recipe (`recipes/qwen3-vl-32b-fp8/`) — see `whitepaper/dynamo/09-global-planner.md` and `10-recipe-qwen3-vl-hetero-xpu-gpu.md`.

❌ Not supported — no `deploy/xpu/` Kubernetes template exists for any of these

None of the 16 cases below have a Kubernetes XPU template today. None are blocked by DRA itself — Intel already has its own DRA driver (`gpu.intel.com`), used by all 8 supported XPU templates. They split into two groups by *why*.

Group A — in progress, not a vendor lock-in

Case	Why not supported
`agg_failover`	Depends on the GMS (GPU Memory Service) sidecar. GMS's own design doc (`lib/gpu_memory_service/GMS_MULTI_DEVICE.md`) shows CUDA support done and an Intel XPU backend (XpuVMM) planned as "Phase 2," just not implemented yet. Not a hardware lock-in, just unfinished
`agg_gms`	Same root cause as `agg_failover`
`gms-failover`	Same root cause, inter-pod failover variant

Group B — no hardware blocker found; simply not yet built (real gap, worth closing)

Case	Why not supported
`agg_kvbm`	Intel XPU support for KVBM is real, active work — not yet merged. Two open GitHub PRs (#7946, #10520, both unmerged as of this writing) add SYCL/Level-Zero KVBM XPU support, tracked by DEP issue #9313 ("kvbm-v2 xpu-sycl enablement"), still in design-discussion stage. Once merged, this K8s template gap should close quickly
`disagg_kvbm`	Same KVBM caveat as above
`disagg_kvbm_2p2d`	Same KVBM caveat as above
`disagg_kvbm_tp2`	Same KVBM caveat as above
`disagg-multinode`	Uses the same NIXL prefill/decode split already proven working on Intel XPU single-node (`deploy/xpu/disagg_xpu_dra.yaml`) — just spread across nodes. No hardware-specific field found; nobody has built the multi-node XPU variant yet
`agg_lora`	No K8s XPU template exists, **but** the equivalent bash launch script already runs on Intel XPU (`examples/backends/vllm/launch/lora/xpu/agg_lora_xp` — strong evidence this is a pure templating gap, not a hardware limitation
`agg_qwen_lora`	Same reasoning as `agg_lora` (multimodal LoRA variant) — no XPU-specific dependency identified, just not templated for K8s yet

Case	Why not supported
`lora-model`	Plain `DynamoModel` CRD for registering a LoRA adapter — hardware-agnostic manifest, no accelerator-specific field; simply hasn't been paired with an XPU deployment example
`minio-secret`	Plain Kubernetes `Secret` for MinIO credentials — hardware-agnostic, works identically regardless of accelerator
`sync-lora-job`	Plain Kubernetes `Job` that downloads/uploads adapter weights — hardware-agnostic, no GPU/accelerator step in the job itself
`gaie/agg`	Gateway API Inference Extension (EPP/InferencePool) integration — this routing-layer pattern is already proven hardware-agnostic elsewhere; no XPU-specific field found, just not built for this integration point yet
`gaie/disagg`	Same reasoning as `gaie/agg`
`gaie/http-route`	Plain Kubernetes Gateway API `HTTPRoute` manifest — contains no hardware reference at all; trivially reusable with any backend

Summary

	Count
✅ Already supported (real `deploy/xpu/` template exists)	8
🔧 Not supported — in progress (GMS/XpuVMM, planned but not yet built)	3 (`agg_failover`, `agg_gms`, `gms-failover`)
❌ Not supported — no blocker found, just not built yet	13

Additional gap found on review: `v1beta1` CRD migration is incomplete for Intel XPU

The repo is mid-migration from the `nvidia.com/v1alpha1` CRD schema (`services:` map) to `nvidia.com/v1beta1` (`components:` list — a real structural change, not just a version bump). `deploy/README.md` states plainly: "Equivalent `nvidia.com/v1beta1` templates are available under `v1beta1/`" — implying full parity. **For Intel XPU specifically, this is not true**:

API version	XPU templates present
`v1alpha1` (`deploy/xpu/`)	All 8 (agg, agg_tracing, agg_router, agg_router_kv_approx, disagg, disagg_tracing, disagg_planner, disagg_router)
`v1beta1` (`deploy/v1beta1/xpu/`)	Only 2 (`agg_xpu_dra.yaml`, `disagg_xpu_dra.yaml`)

The other 6 Intel XPU templates have **not** been ported to the newer `v1beta1` schema

yet. This whitepaper's Chapter 1 (`whitepaper/dynamo/01-aggregated.md` through `08-disaggregated-kv-router.md`) documents the `v1alpha1` versions (matching what the repo actually ships for all 8 patterns). Readers on newer Dynamo Platform versions should note that `v1beta1` XPU coverage is currently partial.

Also noted in passing: `lora/` and `lora/multimodal/` each ship their own separate `minio-secret.yaml` / `sync-lora-job.yaml` (not shared) — both still fall under the same "hardware-agnostic, just not paired with an XPU example" reasoning as the single rows already listed above, so this doesn't change the Group B classification, just the file count.

Follow-up

The 13 "Group B" cases are the actionable backlog: building and validating any of them on real Intel XPU hardware, then contributing a `deploy/xpu/` template upstream, would close a real gap. The LoRA cases (`agg_lora`, `agg_qwen_lora`) are the lowest-risk starting point since the underlying capability is already proven working on Intel XPU via the bash launch-script path.